9

Concurrency

© 2008 Pearson Education, Inc. All rights reserved.

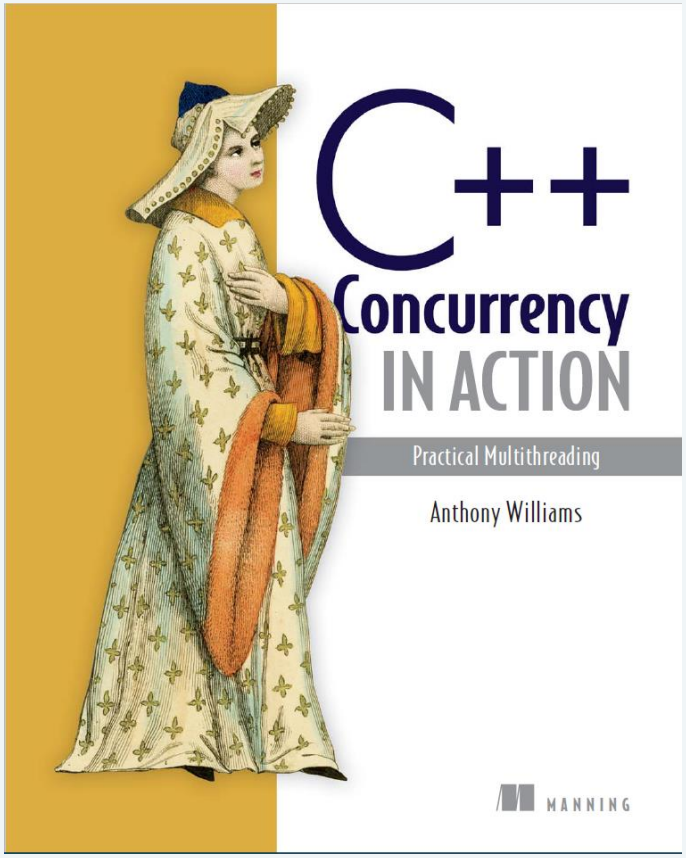
Inhoud

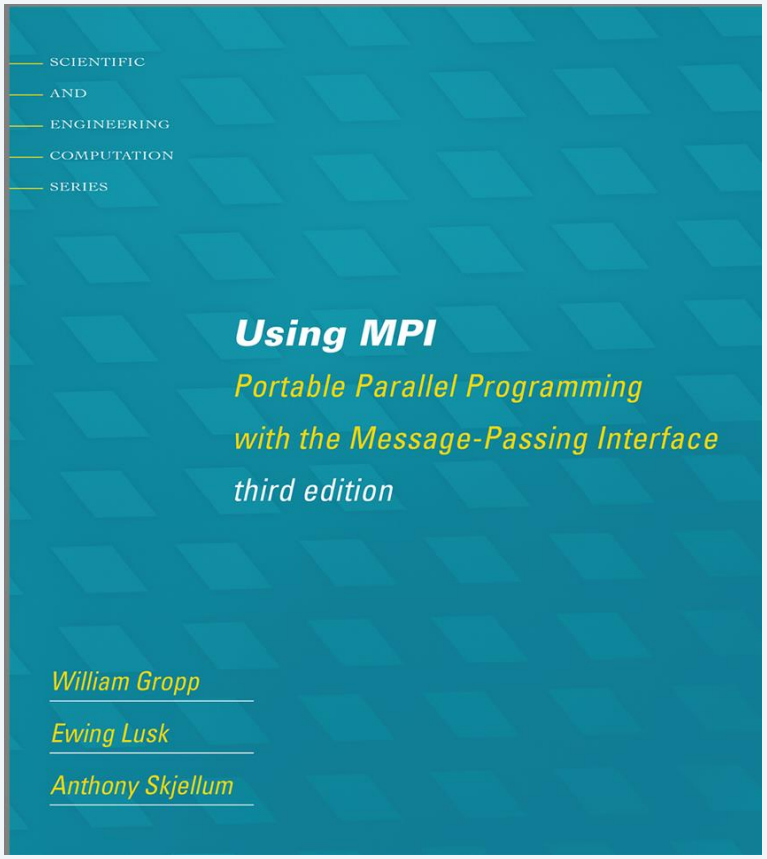
1. Inleiding
2. Programmeren onder Linux
3. Herhaling basisconcepten C++ en OOP
4. Gebruik van STL
5. Externe libraries
6. Inheritance en polymorfisme
7. Templates
8. Object Oriented Design
9. Concurrency
   - 9.1 High performance computing (HPC)
   - 9.2 C++11 Threads
   - 9.3 Message Passing Interface (MPI)
   - 9.4 CalcUA HPC cluster

© 2008 Pearson Education, Inc. All rights reserved.

Referenties

- Door mij gebruikte Boeken:





http://www.mpi-forum.org/docs/docs.html

© 2008 Pearson Education, Inc. All rights reserved.

9.1 High Performance Computing

- Het aggregeren van computing resources ter verhoging van de computationele performantie in vergelijking met een desktop system
- Voor het oplossen van grote computationele problemen in wetenschap, engineering en business
- Historische evolutie: supercomputers → HPC clusters
  - Van gespecialiseerde (dure) hardware naar 'farms' van wijd verkrijgbare commerciële systemen

Cray supercomputer < 1990

© 2008 Pearson Education, Inc. All rights reserved.

9.1.1 HPC computing concepten

- **HPC infrastructuur leent bij uitstek tot parallelisatie en/of concurrency dmv haar multiprocessing architectuur**
- **Multiprocessing: Gebruik van 2 of meer CPU's op een computationeel system**
- **Distributed memory: een multiprocessor systeem waarbij elke CPU geassocieerd is met een eigen geheugen. Berekeningen gebeuren enkel op 'lokale' data.**
- **Shared memory: Dezelfde geheugenruimte wordt gedeeld door verschillende CPU's**
- **Parallellisatie: Het simultaan uitvoeren van verschillende berekeningen: executie van elk process is gelijktijdig**
- **Concurrency: De levensduur van verschillende processen overlapt in de tijd: executie hoeft niet noodzakelijk parallel te verlopen, maar bvb. via time-sharing**
- **Thread: Kleinste reeks programma instructies die onafhankelijk kan behandeld worden door het operating system (scheduler), en is onderdeel van een lopend uitvoerbaar programma (process)**
- **Multithreading: gebruik van uitvoerende programa dat meerdere threads bevat. Threads delen de resources van het process, maar kunnen onafhankelijk worden uitgevoerd.**

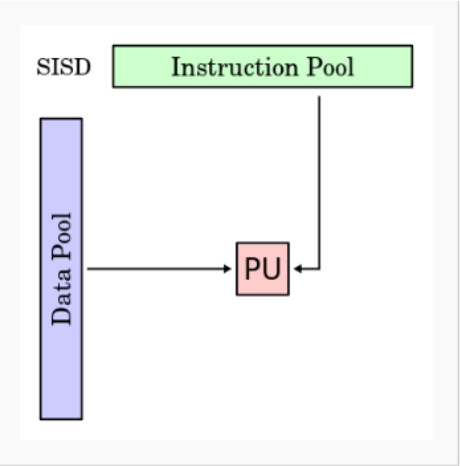
Flynn's taxonomy (multiprogramming context)

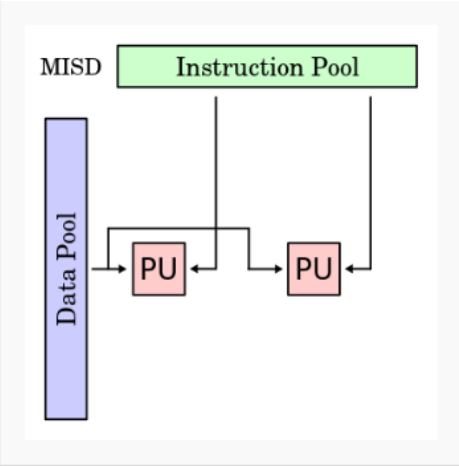
	Single instruction stream	Multiple instruction streams	Single program	Multiple programs
Single data stream	SISD	MISD		
Multiple data streams	SIMD	MIMD	SPMD	MPMD

© 2008 Pearson Education, Inc. All rights reserved.

9.1.2 Flynn Taxonomie

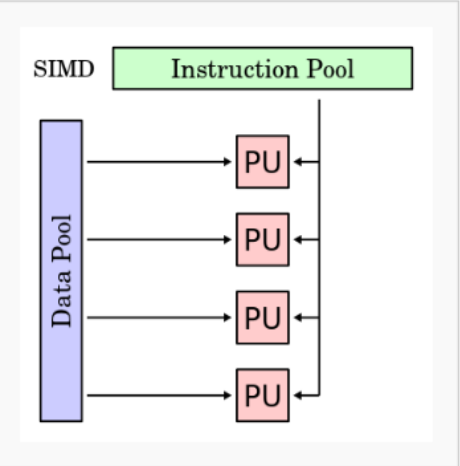
•Uniprocessor, unicore

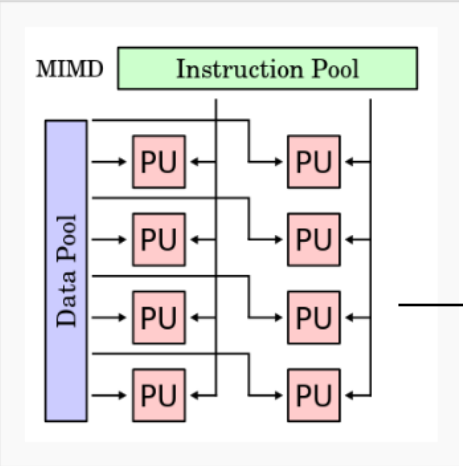




•Pipeline architecture

• GPU, …





• Multicore, distributed systems

•Meest gebruikelijke HPC model

Verfijning van de MIMD

- SPMD:Meerdere autonome processoren die simultaan hetzelfde programma uitvoeren, maar eventueel op andere instructie-posities
- MPMD: Meerdere autonome processoren die minstens 2 onafhankelijke programma's uitvoeren.

© 2008 Pearson Education, Inc. All rights reserved.

9.2 C++11 Threading

- Vroeger POSIX threads (externe bibliotheek)
- Ook in Boost::thread
- Nu opgenomen in de C++ standard C++11
- Een nieuw onderdeel van de C++ standard:
  - Portability en onderhoud
  - Maar hoofdzakelijk low-level features ter implementatie van concurrent code
- Leent zich bij uitstek binnen het shared memory model: verschillende processen hebben allen toegang tot hetzelfde fysieke of virtuele geheugen
- Voor distributed systems: systemen waarbij verschillende hardware (nodes met verschillende CPU cores en geheugen) is verbonden via een snel network is Message Passing meer geschikt (zie later)
- Implementaties van algoritmes kunnen niet altijd zonder susbstantiele aanpassingen worden geparalleliseerd
- Paralellisatie wordt voornamelijk uitgevoerd indien:
  - Deeltaken duidelijk kunnen worden gescheiden (separation of responsabilities)
  - Rekentijd van numerieke gedeelte van je code de limiterende factor is
- Let OP: rekentijd hoeft niet enkel via paralellisatie te worden gereduceerd: efficient gebruik van geheugen (cache) en slimme implementatie kunnen de performantie ook verhogen

© 2008 Pearson Education, Inc. All rights reserved.

9.2.1 Eerste thread voorbeeld

```
#include <iostream>
#include <thread>
void hello()
 {
    std::cout << "Hello threaded  world" << std::endl;
 }
int main(int argc, char *argv[])
{
  std::thread t(hello);
  t.join();


  return 0;
}
```

std thread header

Maak std::thread object en initialiseer met functie die in de thread wordt uitgevoerd

De main thread wacht tot thread t is uitgevoerd

LET op bij compilatie, gebrijk juiste compiler opties!

```
g++ -std=c++11 -pthread -o hellothread hellothread.cpp
```

Voor elke thread moet een std::thread object gemaakt worden (behalve voor de main() thread)

Elke thread heeft een initiële functie nodig (kan member functie van een klasse zijn, zie later)

Na het starten van de neiuwe thread loot de main thread door.

De main thread kan eindigen voor dat thread t zijn werk heft uitgevoerd, daarom wordt de join() member functie aangeroepen: de calling thread (main()) moet dan wachten tot de gemaakte thread t zijn taak heft beëindigd.

9.2.2 Meerdere threads

```
#include<iostream>
#include<thread>

static const int num_threads = 10;
void call_from_thread(int tid)
{
  std::cout << "Launched by thread " << tid << std::endl;
}
int main(int argc, char *argv[])
{
  std::thread t[num_threads];

  //Launch a group of threads
  for (int i = 0; i < num_threads; ++i) {
    t[i] = std::thread(call_from_thread, i);
  }

  std::cout << "Launched from the main\n";

  //Join the threads with the main thread
  for (int i = 0; i < num_threads; ++i) {
    t[i].join();
  }

  return 0;
}
```

Functie die in thread wordt uitgevoerd heeft een argument

Alle threads worden voor gebruiksgemak in een data structuur gestopt

Argument voor de initiële functie wordt meegegeven als argument aan de thread constructor. LET op: dit argument wordt gekopieerd!

© 2008 Pearson Education, Inc. All rights reserved.

9.2.2 Meerdere threads

```
Launched by thread 6
Launched by thread 3
Launched by thread 2
Launched by thread 0
Launched by thread 7
Launched by thread 8
Launched from the main
Launched by thread 4
Launched by thread 5
student@ubuntu-ELD:~/FirstMPI$ ./multiplethreads
Launched by thread 1
Launched by thread 3
Launched by thread 4
Launched by thread 5
Launched by thread 6
Launched by thread 0
Launched by thread 7
Launched by thread 2
Launched from the main
Launched by thread 8
Launched by thread 9
student@ubuntu-ELD:~/FirstMPI$ ./multiplethreads
Launched by thread Launched by thread 3
Launched by thread 0
Launched by thread 4
Launched by thread 1
2
Launched by thread 5
Launched from the main
Launched by thread 6
Launched by thread 7
Launched by thread 9
Launched by thread 8
student@ubuntu-ELD:~/FirstMPI$
```

- **A priori geen vaste volgorde waarin threads worden uitgevoerd: alle 11 threads gaan in competite voor de resources van je system (race-conditie)**
- **Threads wachten ook niet tot andere threads uit hun groep zijn uitgevoerd (synchronisatie)**
- **Geen protectie die garandeert dat meerdere threads dezelde data manipuleren (non-locking)**

© 2008 Pearson Education, Inc. All rights reserved.

9.2.2 Race conditie

```
std::vector<int> bounds(int, int);

void dot_product(const std::vector<int> &v1,const std::vector<int> &v2,
                 int &result, int L, int R)
{
  for (int i=L; i<R; ++i) {
    result+= v1.at(i)*v2.at(i);
  }
}
int main(int argc, char *argv[])
{
  const int nr_elements = 100000;
  const int nr_threads = 20;
  int result = 0;
  std::vector<std::thread> threads;

  // Fill two vectors with constant values for quick verification
  // v1={1,1,1,1, ..., 1}
  // v2={2,2,2,2, ..., 2}
  // The result of the dot product should be 200000
  std::vector<int> v1(nr_elements, 1), v2(nr_elements,2);

  // Split nr-elements into nr_threads parts
  std::vector<int> limits =bounds(nr_threads, nr_elements);

  //Launch nr_threads threads:
  for (int i = 0; i < nr_threads; ++i) {
    threads.push_back(std::thread(dot_product, std::ref(v1), std::ref(v2),
                                  std::ref(result), limits[i], limits[i+1]));
  }


    //Join the threads with the main thread
    for(auto &t : threads){
        t.join();
    }

    //Print the result
    std::cout<<result<<std::endl;


  return 0;
}
```

Berekent inproduct van 2 vectors en stopt resultaat in reference naar integer

Maak vector van threads

Verdeel elementen van v1 en v2 in 20 gelijke segmenten en bepaal de indices van de grenzen via zelf geschreven bounds functie (niet getoond)

Laat elke thread een deel van het inproduct uitrekenen en het resultaat aanpassen

Cast v1 en v2 naar een std::ref om hun reference door te geven en geen kopij van hun reference !!!

© 2008 Pearson Education, Inc. All rights reserved.

9.2.2 Race conditie

```
student@ubuntu-ELD:~/FirstMPI$ ./racingthreads
163124
student@ubuntu-ELD:~/FirstMPI$ ./racingthreads
146818
student@ubuntu-ELD:~/FirstMPI$ ./racingthreads
107206
student@ubuntu-ELD:~/FirstMPI$ ./racingthreads
194208
student@ubuntu-ELD:~/FirstMPI$ ./racingthreads
191070
student@ubuntu-ELD:~/FirstMPI$ ./racingthreads
128714
student@ubuntu-ELD:~/FirstMPI$
```

- **Resultaat klopt niet met verwachting en is ook verschillend bij elke uitvoering**
- **Alle threads proberen gelijktijdig toegang te krijgen tot de reference result (race conditie)**
- **Sommige threads zijn niet in staat om result aan te passen omdat een andere thread bezig is dit te doen**
- **Probleem is te vermijden door het programma te dwingen om de toegant tot variable result te synchroniseren**
- **Mechanisme: mutex (mutual exclusion)**

© 2008 Pearson Education, Inc. All rights reserved.

9.2.2 Mutexes en locking

```
#include <mutex>

std::vector<int> bounds(int, int);
static std::mutex barrier;

void dot_product(const std::vector<int> &v1, const std::vector<int> &v2, int &result, int L, int R){
        int partial_sum = 0;
        for(int i = L; i < R; ++i){
                partial_sum += v1[i] * v2[i];
        }
        std::lock_guard<std::mutex> block_threads_until_finish_this_job(barrier);
        result += partial_sum;
}

int main(){
        int nr_elements = 100000;
        int nr_threads = 20;
        int result = 0;
        std::vector<std::thread> threads;

        //Fill two vectors with some values
        std::vector<int> v1(nr_elements,1), v2(nr_elements,2);

        //Split nr_elements into nr_threads parts
        std::vector<int> limits = bounds(nr_threads, nr_elements);

        //Launch nr_threads threads:
    for (int i = 0; i < nr_threads; ++i) {
        threads.push_back(std::thread(dot_product, std::ref(v1), std::ref(v2), std::ref(result),
                                      limits[i], limits[i+1]));
    }


    //Join the threads with the main thread
    for(auto &t : threads){
        t.join();
    }

    //Print the result
    std::cout<<result<<std::endl;

        return 0;
}
```

Creatie van een global mutex object

Lock_guard is een object dat een mutex locked op moment van creatie en unlocked op moment van destructive

Je kan ook expliciet barrier.lock() en barrier.unlock() aanroepen

Verdeel elementen van v1 en v2 in 20 gelijke segmenten en bepaal de indices van de grenzen via zelf geschreven bounds functie (niet getoond)

Laat elke thread een deel van het inproduct uitrekenen en het resultaat aanpassen

Cast v1 en v2 naar een std::ref om hun reference door te geven en geen kopij van hun reference !!!

© 2008 Pearson Education, Inc. All rights reserved.

9.2.2 Mutexes en locking

```
student@ubuntu-ELD:~/FirstMPI$ ./mutexes
200000
student@ubuntu-ELD:~/FirstMPI$ ./mutexes
200000
student@ubuntu-ELD:~/FirstMPI$ ./mutexes
200000
student@ubuntu-ELD:~/FirstMPI$ ./mutexes
200000
student@ubuntu-ELD:~/FirstMPI$ ./mutexes
200000
student@ubuntu-ELD:~/FirstMPI$ ./mutexes
200000
student@ubuntu-ELD:~/FirstMPI$
```

- **Mutex object is een manier om threads te dwingen te wachten tot een concurrerende thread klaar is met het aanpassen van data**
- **Mutex kunnen gelocked en geopend worden via member functies lock() en unlock()**
- **Elke gelockte mutex moet ook weer unlocked worden, zoniet wachten andere treads oneindig lang op toegang**
- **lock_guard is een handig object dat als hulpmiddel kan gebruikt worden om een mutex automatisch te locken en te unlocken (bij creatie en destructive van het lock_guard object)**
- **Let op wanneer je een mutex locked en unlocked: lock de mutex enkel wanneer er risico bestaat dat treads dezelde data aan passen en lock hem zoalg dit risico bestaat.**

© 2008 Pearson Education, Inc. All rights reserved.

9.3 Message Passing Interface

- **Het Message passing Model:**
  - **Processen uitegvoerd in parallel gebruiken hun eigen address space (deel van het geheugen of fysiek gescheiden geheugen)**
  - **Processen communicern met elkaar door data vanuit de ene address space to kopieren naar de address space van een ander process**
  - **Communicatie verloopt via een vastgelegd protocol dat omvat is in een externe C-library: de MPI library**
  - **Elk process moet controle kunnen uitoefenen over de messages die het krijgt: matching**
  - **Inkomende messages moeten wachten op behandeling: queuing**
  - **MPI behandelt op transparante manier de overdracht van non-contiguous data (data die niet op opeenvolgende adressen is getockeerd), alsook verschillen in data representative tussen verschillende machines (aantal gebruikte bits for gegeven data type)**
  - **Send en receive kunnen blocking zijn (synchronous mode), of non-blocking**
  - **Naast inter-process communicatie is er ook de mogelijkheid van collectieve communicatie (via broadcasts voor data movement en via collective computations zoals sum, min, max ,…)**
  - **6 essentiële functionaliteiten van MPI (plus nog tientallen andere): voor meer info zie http://www.mpi-forum.org/docs/docs.html**

`MPI_Init`	Initialize MPI
`MPI_Comm_size`	Find out how many processes there are
`MPI_Comm_rank`	Find out which process I am
`MPI_Send`	Send a message
`MPI_Recv`	Receive a message
`MPI_Finalize`	Terminate MPI

© 2008 Pearson Education, Inc. All rights reserved.

9.3.1 Eerste MPI Voorbeeld

```
#include <stdio.h>
#include <unistd.h>
#include "mpi/mpi.h"

int main(int argc, char *argv[])
{
int tid,nthreads;
char cpu_name[80];

  /* add in MPI startup routines */
  /* 1st: launch the MPI processes on each node */
  MPI_Init(&argc,&argv);

  /* 2nd: request a thread id, sometimes called a "rank" from
          the MPI master process, which has rank or tid == 0
   */
  MPI_Comm_rank(MPI_COMM_WORLD, &tid);

  /* 3rd: this is often useful, get the number of threads
          or processes launched by MPI, this should be NCPUs-1
   */
  MPI_Comm_size(MPI_COMM_WORLD, &nthreads);

  /* get the machine name of this particular host ... well
     at least the first 80 characters of it ... */
  gethostname(cpu_name,80);

  printf("hello MPI user: from process = %i on machine=%s, of NCPU=%i processes\n",
         tid, cpu_name, nthreads);
  MPI_Finalize();
  return 0;

}
```

Include de MPI library (plaats=system afhankelijk)

MPI initaliastie: mag slechts 1 keer voorkomen per uitvoerbaar programma

Vraag de ‘rang’ van de thread op

MPI_COMM_WORLD = default MPI communicator

Vraag het total aantal threads op

Verkrijg de naam van de processor/system/node waar het process wordt uitgevoerd

Verplicht aan einde van elk uitvoerbaar programma

on, Inc. All rights reserved.

9.3.1 Eerste MPI Voorbeeld

```
student@ubuntu-ELD:~/Concurrency$ mpiCC -o hellompi hellompi.c
student@ubuntu-ELD:~/Concurrency$ mpirun -np 4 hellompi
hello MPI user: from process = 1 on machine=ubuntu-ELD, of NCPU=4 processes
hello MPI user: from process = 0 on machine=ubuntu-ELD, of NCPU=4 processes
hello MPI user: from process = 3 on machine=ubuntu-ELD, of NCPU=4 processes
hello MPI user: from process = 2 on machine=ubuntu-ELD, of NCPU=4 processes
student@ubuntu-ELD:~/Concurrency$
```

- **Let op compilatie: mpicc (voor c-code), mpiCC (voor C++ code)**
- **Aantal threads (in MPI processes) wordt bepaald bij uitvoering van het programma, niet bij implementatie: mpirun –np #aantal processen <programma>**
- **Volgorde waarin processen worden uitgevoerd ligt niet a-priori vast (zie C++11 threads)**
- **MPI_Init dient verplicht en als eerste worden aangeroepen, mag slechts 1 keer worden aangeroepen per uitvoerbaar programma**
- **Argumenten van MPI_init zijn de adressen van argc en argv (de parameters die bij uitvoering aan het hoofprogramma kunnen worden meegegeven)**
- **Communicatie tussen processen wordt verzorgd door een ‘communicator’ object waarmee een groep van processen geacossocieerd wordt**
- **Elk process behorende bij een communicator heft een ‘rang’ die loop van 0, …, nproc-1**
- **De default communicator heet MPI_COMM_WORLD, de communicator die alle processen van de MPI applicatie beheert**

© 2008 Pearson Education, Inc. All rights reserved.

9.3.2 Point-to-point communicatie tussen processen

```
int main(int argc, char *argv[])
{
  int tid,nthreads;
  MPI_Status stat;
  int nchar = 80;
  char message_sent[nchar], message_recieved[nchar];
  // Setup default message
  strcpy(message_sent, "No message sent");
  strcpy(message_recieved, "No message recieved");

  MPI_Init(&argc,&argv);
  MPI_Comm_rank(MPI_COMM_WORLD, &tid);
  MPI_Comm_size(MPI_COMM_WORLD, &nthreads);

  if (nthreads>1) {
    if (tid==0) {
      strcpy(message_sent, "Hello from process 0");
      MPI_Send(message_sent, strlen(message_sent)+1, MPI_CHAR, 1, 1, MPI_COMM_WORLD);
      MPI_Recv(message_recieved, nchar, MPI_CHAR, 1, 1, MPI_COMM_WORLD, &stat);
    } else if (tid==1) {
      MPI_Recv(message_recieved, nchar, MPI_CHAR, 0, 1, MPI_COMM_WORLD, &stat);
      strcpy(message_sent, "Process 1 got this message: ");
      strcat(message_sent ,message_recieved);
      MPI_Send(message_sent, strlen(message_sent)+1, MPI_CHAR, 0, 1, MPI_COMM_WORLD);
    }
    printf("Process %i sent %s\n", tid, message_sent);
    printf("Process %i recieved %s\n", tid, message_recieved);

  } else {
    printf("Not enough processes to demo message passing\n");
  }
```

Process met rank 0 stuurt data naar process met rank 1 via MPI_Send die 6 argumenten krijgt:

Adres van een buffer die de te versturen data bevat, de hoeveelheid elementen in de buffer, het data type, rangnummer van de geadresseerde, een ‘tag’ en de te gebruiken communicator

© 2008 Pearson Education, Inc. All rights reserved.

9.3.2 Point-to-point communicatie tussen processen

```
student@ubuntu-ELD:~/Concurrency$ mpirun -np 2 mpisendrecieve
Process 0 sent Hello from process 0
Process 0 recieved Process 1 got this message: Hello from process 0
Process 1 sent Process 1 got this message: Hello from process 0
Process 1 recieved Hello from process 0
student@ubuntu-ELD:~/Concurrency$
```

- **Bij oproep an MPI_Send wordt een message buffer in het geheugen aangemaakt die naast de te versturen data een 'message envelope' bevat**
- **De message envelope bevat de rank van de source, de receiver, de communicator en een 'tag' (integer die toelaat aan de receiver om selectief messages te lezen of te negeren)**
- **MPI_Send en Receive functies krijgen een MPI datatype argument (omdat op gedistribueerde systemen het aantal bytes voor een gegeven datatype kan verschillen)**
- **Naast het address van de databuffer specifieert men het aantal elementen dat van een gegeven MPI datatype dient te worden gekopieerd (men kan dus arrays van data doorsturen)**
- **De lengte van de receive buffer moet groot genoeg zijn om de data te ontvangen**
- **MPI_Recv heft nog een extra argument van het type MPI_Stat: een data struct met 3 data velden (MPI_SOURCE, MPI_TAG en MPI_ERROR, die informatie bevat over het al dan niet slagen van de receive operatie**
- **Naast specifieke identifiers voor de source en de tag kunnen wildcards gebruikt worden: MPI_ANY_SOURCE en MPI_ANY_TAG**
- **Een receive operatie kan messages accepteren van een arbitraire sender, maar een sender moet een specifieke receiver opgeven**

© 2008 Pearson Education, Inc. All rights reserved.

9.3.2 Point-to-point communicatie tussen processen

- **Communicatie Modes:**
  - **De MPI_Send functie is ‘blocking’: ze wordt niet beeindigd voordat de send buffer en de bijbehorende envelope zijn gestockeer in het gehuegen**
  - **De message kan direct in de buffer van de ontvange gekopieerd worden of in een tijdelijke systeembuffer (MPI heeft hier de vrijheid om zelf die keuze te maken)**
  - **Indien de send data gebufferd wordt, kan de MPI_Send worden beeindigd nog voordat een MPI_Recv oproep de buffer heeft ontvangen**
  - **Men kan MPI dwingen om de send data te bufferen via oproep van MPI_Bsend (op die manier kan send altijd beeindigd worden zonder dat er een ovreenkomstige receive is gestart)**
  - **Men kan MPI dwingen om een send uit te voeren in ‘synchronous’ mode via oproep van MPI_Ssend (op die manier dwingt men send te wachten tot er een overeenkomstige receive is geinitieerd)**
  - **De meest stricte send mode is de ‘ready’ mode via oproep van MPI_Rsend , waarbij een send operatie enkel kan starten nadat er al een receive operatie is geninitieerd**
  - **Er is maar 1 sort MPI_Recv functie die qua mode de overeenkomstige Send opdracht matched**
  - **MPI_Recv is ook ‘blocking’: De fundtie wordt enkel beeindigd wanner de receive buffer is opgevuld**
  - **MPI_Recv kan beindigen voordat een overeenkomstige send is beeindigd, maar niet voordat een ovreenkomstige send is gestart**

© 2008 Pearson Education, Inc. All rights reserved.

9.3.2 Point-to-point communicatie tussen processen

- **Messages zijn ‘non-overtaking’:**
  - **Indien een sender twee messages stuurt naar dezelfde receiver, dan kan de receiver de tweede message enkel ontvangen nadat de eerste receive beeindigd is**
  - **Indien een receiver twee receives oproept die verwijzen naar dezelde message, dan kan de tweede receive opdracht niet worden uitgevoerd**
- **Bij multithreaded processen is de volgeorde waarin sends en receives worden uitgevoerd vanuit verschillende processen niet deterministisch**
- **Elke communicatie opdracht gebruikt locale resources (geheugen en CPU), indien er niet voldoende resources aanwezig zijn onstaan er errors en/of ongedefinieerd gedrag**
- **Dit laatste problem kan worden opgevangen door een buffered send uit te voeren met een expliciete geheugen allocatie voor de message buffer, indien er niet voldoende geheugen beschikbaar is kan de send niet beeindigen en zal er een error code worden gegenereerd.**
- **Van elke MPI send en receive functies bestaan er ook non-blocking versies: MPI_Isend, MPI_Ibsend, MPI_Issend, MPI_Irsend, en MPI_Irecv.**
- **Men kan non-blocking sends combineren met blocking receives en vice-versa**
- **Non-blocking communicatie wordt afgehandeld via de MPI_Wait en MPI_Test functies**

© 2008 Pearson Education, Inc. All rights reserved.

9.3.3 MPI Datatypes

MPI datatype	C datatype
MPI_CHAR	char (treated as printable character)
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_LONG_LONG_INT	signed long long int
MPI_LONG_LONG (as a synonym)	signed long long int
MPI_SIGNED_CHAR	signed char (treated as integral value)
MPI_UNSIGNED_CHAR	unsigned char (treated as integral value)
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_UNSIGNED_LONG_LONG	unsigned long long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_WCHAR	wchar_t (defined in <stddef.h>) (treated as printable character)
MPI_C_BOOL	_Bool
MPI_INT8_T	int8_t
MPI_INT16_T	int16_t
MPI_INT32_T	int32_t
MPI_INT64_T	int64_t
MPI_UINT8_T	uint8_t
MPI_UINT16_T	uint16_t
MPI_UINT32_T	uint32_t
MPI_UINT64_T	uint64_t
MPI_C_COMPLEX	float _Complex
MPI_C_FLOAT_COMPLEX (as a synonym)	float _Complex
MPI_C_DOUBLE_COMPLEX	double _Complex
MPI_C_LONG_DOUBLE_COMPLEX	long double _Complex
MPI_BYTE	
MPI_PACKED	

Naast het versturen van buffers van data elementen van zelfde type die op opeenvolgende geheugenlocaties zijn opgeslagen, bidet MPI de mogelijkheid om data buffers met gemengde types en met niet contiguous data te versturen: hier niet behandeld

© 2008 Pearson Education, Inc. All rights reserved.

9.3.4 Collectieve Communicatie

```
#include <iostream>
#include <vector>
#include "mpi/mpi.h"

int main(int argc, char *argv[])
{
  int tid,nthreads;

  int nr_elements;
  int result = 0;

  MPI_Init(&argc,&argv);
  MPI_Comm_rank(MPI_COMM_WORLD, &tid);
  MPI_Comm_size(MPI_COMM_WORLD, &nthreads);

  // ask users to provide a number of elements
   while (1)
    {
      if (tid==0) {
        std::cout << "enter the number of elements (use 0 to quit): ";
        //scanf("\%d", &nr_elements);
        std::cin >> nr_elements;
      }
      MPI_Bcast(&nr_elements, 1, MPI_INT, 0, MPI_COMM_WORLD);
      //Broadcast the nr of elements to all processes

      if (nr_elements == 0)
                break;
      else {
          //Fill two vectors with some values
        std::vector<int> v1(nr_elements,1), v2(nr_elements,2);
```

Stuur waarde van nr_elements naar alle processen die onder de COMM_WORLD communicator vallen

© 2008 Pearson Education, Inc. All rights reserved.

9.3.4 Collectieve Communicatie

```
      //Compute the dot product for each chunk
      int partial_sum = 0;

      int delta = nr_elements / nthreads;
      int START = tid*delta;
      int STOP = (tid+1)*delta;
      if (tid==(nthreads-1))
        STOP = nr_elements;
      std::cout << "nr_elements: "<< nr_elements<< " START: "<< START << "  STOP:  " << STOP << std::endl;
      for(int i = START; i < STOP; ++i){
        partial_sum += v1[i] * v2[i];
      }
      std::cout << "Partial sum is  " << partial_sum << std::endl;


      MPI_Reduce(&partial_sum, &result, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

      if (tid==0) std::cout << "the product is: " << result << std::endl;
      }
    }
  MPI_Finalize();
  return 0;
}
```

Munch … munch

Haal alle waarden van partial_sum op van alle processen zodra ze klaar zijn, sommeer ze en stop het resultaat in result

© 2008 Pearson Education, Inc. All rights reserved.

9.3.4 Collectieve Communicatie

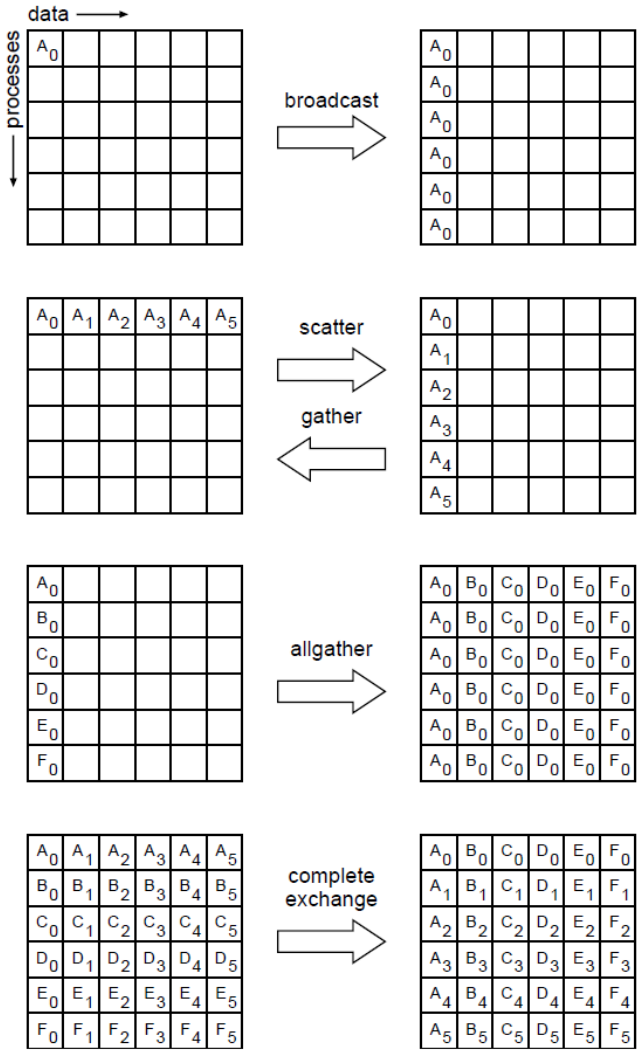
```
student@ubuntu-ELD:~/Concurrency$ mpirun -np 10 ./dotproductmpi
enter the number of elements (use 0 to quit): 10000000
nr_elements: 10000000 START: 8000000  STOP:  9000000
nr_elements: 10000000 START: 1000000  STOP:  2000000
nr_elements: 10000000 START: 5000000  STOP:  6000000
nr_elements: 10000000 START: 0  STOP:  1000000
nr_elements: 10000000 START: 2000000  STOP:  3000000
nr_elements: 10000000 START: 4000000  STOP:  5000000
Partial sum is  2000000
nr_elements: 10000000 START: 6000000  STOP:  7000000
nr_elements: 10000000 START: 7000000  STOP:  8000000
nr_elements: 10000000 START: 3000000  STOP:  4000000
Partial sum is  2000000
Partial sum is  2000000
Partial sum is  2000000
nr_elements: 10000000 START: 9000000  STOP:  10000000
Partial sum is  2000000
Partial sum is  2000000
Partial sum is  2000000
Partial sum is  2000000
Partial sum is  2000000
Partial sum is  2000000
the product is: 20000000
enter the number of elements (use 0 to quit): 0
student@ubuntu-ELD:~/Concurrency$
```

© 2008 Pearson Education, Inc. All rights reserved.

9.3.4 Collectieve Communicatie

- **Bij collectieve communicatie worden send en receive opdrachten gestuurd naar alle processen die bij dezelfde communicator horen, i.e. alle processen indien MPI_COMM_WORLD de communicator is**
- **Verschillende mogelijkheden:**

- MPI_BARRIER, MPI_IBARRIER: Barrier synchronization across all members of a group (Section 5.3 and Section 5.12.1).
- MPI_BCAST, MPI_IBCAST: Broadcast from one member to all members of a group (Section 5.4 and Section 5.12.2). This is shown as "broadcast" in Figure 5.1.
- MPI_GATHER, MPI_IGATHER, MPI_GATHERV, MPI_IGATHERV: Gather data from all members of a group to one member (Section 5.5 and Section 5.12.3). This is shown as "gather" in Figure 5.1.
- MPI_SCATTER, MPI_ISCATTER, MPI_SCATTERV, MPI_ISCATTERV: Scatter data from one member to all members of a group (Section 5.6 and Section 5.12.4). This is shown as "scatter" in Figure 5.1.
- MPI_ALLGATHER, MPI_IALLGATHER, MPI_ALLGATHERV, MPI_IALLGATHERV: A variation on Gather where all members of a group receive the result (Section 5.7 and Section 5.12.5). This is shown as "allgather" in Figure 5.1.
- MPI_ALLTOALL, MPI_IALLTOALL, MPI_ALLTOALLV, MPI_IALLTOALLV, MPI_ALLTOALLW, MPI_IALLTOALLW: Scatter/Gather data from all members to all members of a group (also called complete exchange) (Section 5.8 and Section 5.12.6). This is shown as "complete exchange" in Figure 5.1.
- MPI_ALLREDUCE, MPI_IALLREDUCE, MPI_REDUCE, MPI_IREDUCE: Global reduction operations such as sum, max, min, or user-defined functions, where the result is returned to all members of a group (Section 5.9.6 and Section 5.12.8) and a variation where the result is returned to only one member (Section 5.9 and Section 5.12.7).
- MPI_REDUCE_SCATTER_BLOCK, MPI_IREDUCE_SCATTER_BLOCK, MPI_REDUCE_SCATTER, MPI_IREDUCE_SCATTER: A combined reduction and scatter operation (Section 5.10, Section 5.12.9, and Section 5.12.10).
- MPI_SCAN, MPI_ISCAN, MPI_EXSCAN, MPI_IEXSCAN: Scan across all members of a group (also called prefix) (Section 5.11, Section 5.11.2, Section 5.12.11, and Section 5.12.12).



© 2008 Pearson Education, Inc. All rights reserved.

9.4 Conclusie

- **C++11 threads en de MPI library zijn allebei valabele tools voor paralellisatie van een numeriek problem op een cluster**
- **MPI bevat veel meer functionaliteiten van een hoger abstractie niveau dan C++11 threading**
- **MPI bevat in de laatste standaard geen C++ bindings meer, een deel van je C++ code die gebruik maakt van MPI zal dus C-style arrays en function calls moeten gebruiken**
- **We behandelen hier enkel de basisfunctionaliteiten van paralellsisatie via C++11 en MPI**
- **Parallel en concurrent numerical computing, HPC computing etc. zijn gespecialiseerde domeinen in de numerieke wiskunde, informatica**
- **Toch zijn een groot aantal fysische prolemen eenvoudig te paralelliseren**
- **Maak je vertrouwd met de basisbegrippen en verdiep je verder via zelfstudie indien nodig in je latere carrier**
- **Opdracht: paralelliseer een numerieke benedering van een eenvoudige integraal**

© 2008 Pearson Education, Inc. All rights reserved.